

Pashov Audit Group

Spectra Security Review

October 27th 2026 - November 1st 2026

Contents

1. About Pashov Audit Group . . . 3
2. Disclaimer . . . 3
3. Risk Classification . . . 3
4. Methodology . . . 4
5. Executive Summary . . . 4
6. Findings . . . 4
Medium Severity . . . 6
[M-01] OneInchLimitOrderModule runs maker order hooks while the router flash-loan authorization is live, so the maker can spend every token the taker approved to the router . . . 6
Low Severity . . . 8
[L-01] Unbounded revert data can defeat `skipReverts` and revert the entire batch . . . 8
[L-02] A single contract maker order can revert a whole best effort batch fill . . . 9
[L-03] Self-registered fake registry row makes SettlementRouter pay fees from its own balance without collecting them, so anyone can drain tokens held by the router . . . 11
[L-04] SettlementRouter redeposits even a sub-share amount into the IBT and reverts on failure, so a 1-wei deposit request blocks settle, forceSettle and forceSettleWithNav . . . 13
[L-05] Per-share NAV rescaling truncates aggregate vault value . . . 15
[L-06] SettlementRouter stores gross NAV after charging settlement fees . . . 16
Info Severity . . . 18
[I-01] The protocol specification rejects the vault share quote kind the contracts accept . . . 18
[I-02] `previewSettle` reports success for settlements that fail the vault's maximum-drawdown check . . . 19
[I-03] Fee-charging ERC-4626 IBTs cause Amphor to record more backing than the Safe owns . . . 20
[I-04] Rate getter prices SELL orders with the IBT's full-amount, liquidity-capped `previewRedeem`: a flash-loan taker buys a signed 1,000,000 PT order on an Aave ATokenVault market for under 2 USDC . . . 21
[I-05] Rate getter fills IBT-quoted BUY orders at a stale stored rate when the IBT preview is capped, so a free PT flash loan lets the taker deliver less PT than the maker's signed APY . . . 24
[I-06] Conservative ERC-4626 preview can falsely fail tokeniser minOut checks . . . 26
[I-07] Oracle settlement accepts reports older than a manual NAV override . . . 27

1. About Pashov Audit Group

Pashov Audit Group consists of 100+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 500 security audits completed - uncovering and helping patch thousands of vulnerabilities - the group strives to create the most robust and pleasant audit experience possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Telegram @pashovkrum.

2. Disclaimer

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue has been reviewed; the outcome is recorded in the finding's Resolution section.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Critical	Loss or compromise of critical assets, or protocol-wide impact.
High	Significant material loss of assets, or serious harm to a group of users.
Medium	Moderate material loss, or moderate harm, often under conditional assumptions.
Low	Minor impact, limited scope, or requires unlikely conditions to trigger.
Info	No direct security impact - best-practice, code-quality or hardening suggestion.

IMPACT

High - leads to a significant material loss of assets in the protocol or significantly harms a group of users

Medium - leads to a moderate material loss of assets in the protocol or moderately harms a group of users

Low - leads to a minor material loss of assets in the protocol or harms a small group of users

LIKELIHOOD

High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost

Medium - only a conditionally incentivized attack vector, but still relatively likely

Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

A large team of security researchers hunt vulnerabilities on the scope provided during a time-boxed review.

Every issue submission is deduplicated and validated by a judging team. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

5. Executive Summary

A time-boxed security review of Spectra was conducted by the Pashov Audit Group team. This report documents 14 issues across the reviewed codebase, reported by ast3ros, Hals, IvanFitro, Kalyan, Said, Shred Security, Slayer Security, Stan.

SCOPE

perspectivefi/spectra-lo-otc, branch main 80e2ee4b563ac34cad4f6e1db87acac626de6873
./src/SpectraPtYtRateAmountGetter.sol
./src/libraries/SpectraRatePricingLib.sol
./src/SpectraAuthorizedTakersGuard.sol
./src/SpectraLimitOrderSigner.sol
./src/SpectraLimitOrderBatchFiller.sol
./src/SpectraLimitOrderTokeniser.sol
./src/SpectraRateFeeTaker.sol

perspectivefi/metavaults-contracts, PR #151 branch feat/final-settlement-router
eb328964982fae2d76e205c95d867afcd4f8638d
./src/SettlementRouter.sol
./src/SafeInfraVaultReader.sol
./src/oracles/ManualNavOracleAdapter.sol

perspectivefi/spectra-router, PR #31 branch feat/one-inch-limit-order-module
d312dcfb371fd00fb36d5fb2de146797ac2e7956
./src/router/modules/OneInchLimitOrderModule.sol

6. Findings

ID	TITLE	SEVERITY	STATUS
M-01	OneInchLimitOrderModule runs maker order hooks while the router flash-loan authorization is live, so the maker can spend every token the taker approved to the router	Medium	Fixed
L-01	Unbounded revert data can defeat `skipReverts` and revert the entire batch	Low	Fixed

L-02	A single contract maker order can revert a whole best effort batch fill	Low	Fixed
L-03	Self-registered fake registry row makes SettlementRouter pay fees from its own balance without collecting them, so anyone can drain tokens held by the router	Low	Fixed
L-04	SettlementRouter redeposits even a sub-share amount into the IBT and reverts on failure, so a 1-wei deposit request blocks settle, forceSettle and forceSettleWithNav	Low	Acknowledged
L-05	Per-share NAV rescaling truncates aggregate vault value	Low	Acknowledged
L-06	SettlementRouter stores gross NAV after charging settlement fees	Low	Fixed
I-01	The protocol specification rejects the vault share quote kind the contracts accept	Info	Fixed
I-02	`previewSettle` reports success for settlements that fail the vault's maximum-drawdown check	Info	Fixed
I-03	Fee-charging ERC-4626 IBTs cause Amphor to record more backing than the Safe owns	Info	Acknowledged
I-04	Rate getter prices SELL orders with the IBT's full-amount, liquidity-capped `previewRedeem`: a flash-loan taker buys a signed 1,000,000 PT order on an Aave ATokenVault market for under 2 USDC	Info	Acknowledged
I-05	Rate getter fills IBT-quoted BUY orders at a stale stored rate when the IBT preview is capped, so a free PT flash loan lets the taker deliver less PT than the maker's signed APY	Info	Acknowledged
I-06	Conservative ERC-4626 preview can falsely fail tokeniser minOut checks	Info	Fixed
I-07	Oracle settlement accepts reports older than a manual NAV override	Info	Fixed

Medium Severity

[M-01] OneInchLimitOrderModule runs maker order hooks while the router flash-loan authorization is live, so the maker can spend every token the taker approved to the router

Medium Fixed

SUMMARY

When a router user sells PT to a 1inch limit order using the new command `FILL_1INCH_LIMIT_ORDERS` (`0x27`) inside a `FLASH_LOAN` batch, the maker's order hook can execute router commands as that user.

`OneInchLimitOrderModule.fillOneInchOrders` passes each `Fill` to `BatchFiller.fillMany` , and 1inch LOP 4.3.2 performs plain `CALL` s into the maker's pre-interaction, post-interaction, and `makerPermit` targets. `SpectraLimitOrderBatchFiller._validate` checks nothing about these maker hooks, and `_targetedTraits` keeps the caller's skip-permit bit. So the maker's order names code that runs inside the batch.

The module guard accepts `msg.sender == address(this)` , and only the flash-loan callback calls `execute` this way, so `0x27` is intended to run inside `FLASH_LOAN` as the replacement for the `CURVE_SWAP(PT->IBT)` leg of the documented B29 flash swap.

```
address sender = LibExecutionModule.getMsgSender();
if (sender == address(0) || (msg.sender != sender && msg.sender != address(this))) {
    revert OnlyDuringExecution();
}
```

`FlashloanModule.onFlashLoan` accepts a callback when `msg.sender` is the lender stored by `flashLoan` and an execution is live. It does not check `initiator` , and Spectra `PrincipalToken.flashLoan` is permissionless with a caller-chosen receiver. The maker hook therefore calls `PT.flashLoan(receiver = router, IBT, 1, commands)` on the same PT the victim borrows from. The router runs `commands` as a self-call, which keeps `msgSender = victim` , so `TRANSFER_FROM` pulls a token the victim approved to the router and `TRANSFER` sends it to the attacker. The 1 wei loan is repaid and the victim's batch continues. The hook identifies the victim by `tx.origin` and takes `min(balance, allowance)` .

AUDIT.md's `onFlashLoan()` spoof mitigation (Spoof.3.R.1) assumes only user-named code (like a Curve pool) runs in the flash-loan window. `0x27` is the first command where the counterparty's signed order names code that runs in the batch. None of the user's checks (price, amounts, `threshold` , `minMakingFilled` , `ASSERT_MIN_BALANCE`) depend on the hook target.

The root cause is in the module: it runs counterparty code while the router still holds a live lender authorization inside `FLASH_LOAN` , without clearing it or restricting the hooks. `onFlashLoan` not checking `initiator` makes it reachable.

IMPACT

The maker of any order a router user fills through `0x27` inside `FLASH_LOAN` can transfer to itself `min(balance, allowance)` of every token that user approved to the router. Order size does not limit the loss.

In the PoC the order is 950 IBT for 1,000 PT yet it takes 250,000 USDC. The victim's transaction succeeds, the order fills at its signed price, and `threshold`, `minMakingFilled`, and `ASSERT_MIN_BALANCE(YT >= 1,000)` all hold, so the trade looks correct to the user.

The exposure is any allowance beyond what the current batch pulls, for example a max approval left from an earlier trade. An exact approval sized to the trade's own input cannot be taken without reverting the victim's transaction. Standing approvals to the Spectra router exist in practice; the July 2024 router incident drained tokens four wallets had approved to it.

The issue is invisible to simulation: outside the flash-loan window the hook does nothing, and a `0x27` fill without `FLASH_LOAN` makes the nested callback revert and be ignored while the fill completes normally.

Off-chain controls do not prevent this. PROTOCOL_SPEC §10/§15/§18 tell the SDK and orderbook to reject orders with a post-interaction or maker permit, but nothing on-chain rejects them. `0x27` is permissionless and its `Fill[]` is not tied to Spectra's book, so a solver sourcing LOP orders elsewhere passes them through. A post-interaction is also normal for legitimate fee orders (`SpectraRateFeeTaker`); a hooked order differs only in the target address.

EXPLOIT PATH

1. Attacker signs a 1inch limit order with a maker pre/post-interaction (or maker permit) pointing at attacker-controlled code.
2. A router user fills this order via `FILL_1INCH_LIMIT_ORDERS` (`0x27`) inside a `FLASH_LOAN` batch (the B29-style flash swap).
3. During fill, LOP calls the maker hook, which calls `PT.flashLoan(receiver = router, IBT, 1, commands)` on the PT the victim is borrowing from.
4. The router runs `commands` as a self-call with `msgSender` still the victim; `TRANSFER_FROM` pulls tokens the victim approved to the router and `TRANSFER` sends them to the attacker.
5. The 1 wei loan repays, the victim's batch completes successfully at the signed price.

PREREQUISITES

No privileged role is needed. The attacker only needs one signed limit order and the gas for its hook, and the victim must have a standing allowance to the router larger than the current batch's input (for example a leftover max approval). `onFlashLoan` not checking `initiator` makes the hijack reachable.

RECOMMENDATION

1. Clear the lender while counterparty code runs. In `fillOneInchOrders`, zero the flash-loan lender before calling `fillMany` and restore it afterward. `0x27` stays usable inside `FLASH_LOAN`, and this also covers code reached through the order's tokens.

```
LibFlashloanModule.FlashloanStorage storage fs = LibFlashloanModule.flashloanStorage();
address lender = fs.flashloanLender;
fs.flashloanLender = address(0);
(totalMaking, filled, spent) =
    ISpectraLimitOrderBatchFiller(BATCH_FILLER).fillMany(fills, budgets, minMakingFilled,
skipReverts);
fs.flashloanLender = lender;
```

2. Restrict maker hooks in `_validate`. Allow a pre-interaction (bit 252) only to the authorized-takers guard and a post-interaction (bit 251) only to `SpectraRateFeeTaker`, held as constructor immutables since the BatchFiller is ownerless. Reject an empty target, since LOP then calls the maker. Read targets with `ExtensionLib.preInteractionTargetAndData` / `postInteractionTargetAndData`. Also set the skip-maker-permit bit (taker traits bit 253) in `_targetedTraits`, or reject a non-empty `makerPermit` (v1 orders never carry one).

3. Defence in depth. Require `initiator == address(this)` in `FlashloanModule.onFlashLoan` (the standard ERC-3156 check).

RESOLUTION

Spectra

We addressed M-02 in https://github.com/perspectivefi/spectra-router/pull/36, which is now merged.

We chose to consume the lender authorization directly in FlashloanModule.onFlashLoan(), immediately after validating the callback and before executing commands. This prevents maker hooks from reusing that authorization without restricting supported hooks. Explicit nested loans still establish their own authorization through the router.

Low Severity

[L-01] Unbounded revert data can defeat `skipReverts` and revert the entire batch

Low Fixed

SUMMARY

`SpectraLimitOrderBatchFiller.fillMany()` exposes a `skipReverts` option that is meant to let the batch continue when one settlement fails:

```
(bool success, uint256 making, bytes memory reason) = _fill(fills[i]);
if (success) {
    // ...
} else if (skipReverts) {
    emit FillSkipped(msg.sender, i, reason);
}
```

The problem is that `_fill()` captures the full revert data returned by the 1inch Limit Order Protocol:

```
} catch (bytes memory err) {
    return (false, 0, err);
}
```

An attacker-controlled maker, signature validator, token, or order interaction can burn most of the forwarded gas and then revert with a very large return payload. Solidity must allocate memory and copy the entire payload into `err` before the catch branch can return, and the BatchFiller then includes that same payload in the `FillSkipped` event, consuming further memory and log-data gas.

The crafted call can leave too little gas to copy and return the revert data, emit `FillSkipped`, process the remaining orders, or run `_close()` to revoke approvals and refund unused budgets. Rather than skipping the bad order, the whole batch reverts.

IMPACT

A single malicious order placed in a best-effort batch can block every other order from executing even though `skipReverts = true` was requested.

All earlier successful fills and budget operations are rolled back and the caller loses the transaction gas. The caller must locate and remove the malicious order before retrying.

There is no partial asset loss, since the whole transaction reverts. The impact is gas griefing and denial of service against any batch that contains the adversarial order.

RECOMMENDATION

Run each settlement through a bounded low-level call that reserves enough gas for failure handling, the remaining fills, approval cleanup, and refunds.

On failure, copy and emit only a fixed-size prefix of the revert data, such as the four-byte selector or the first few hundred bytes. Avoid materializing the complete return buffer as a dynamic Solidity `bytes` value.

RESOLUTION

> **Spectra**
>
> We acknowledge both findings and propose a fix in PR #21:
>
> - **L-01:** skipped fills copy and emit at most the 4-byte error selector, preventing oversized revert data from exhausting the batch's gas.
> - **L-02:** each settlement call has a caller-selected gas cap, covering signature checks and hooks.

[L-02] A single contract maker order can revert a whole best effort batch fill

Low Fixed

SUMMARY

`SpectraLimitOrderBatchFiller.fillMany()` with `skipReverts` set is meant to skip any fill that fails and settle the rest. Each fill is an external call to the settlement that forwards all remaining gas, and for a contract maker the settlement verifies the signature with a `staticcall` to the maker's `isValidSignature()` that again forwards all gas.

A malicious maker contract can read a stored flag: it behaves normally while the taker simulates the batch, then the maker arms the flag after the taker broadcasts. Once armed, `isValidSignature()` loops

until it runs out of gas, so the signature check consumes nearly all the forwarded gas and the settlement call fails.

The batch filler is left with only a few percent of its gas, which is not enough to settle the remaining fills and refund the budgets under any gas limit derived from the estimate, so the whole transaction reverts. Contract makers are a supported order profile, so no unusual extension is needed, and maker pre-interaction or post-interaction contracts receive the same forwarded gas and can do the same.

IMPACT

The taker loses the gas of the whole transaction and every honest fill in the batch, even though it asked for failures to be skipped.

A three order batch that needs about 245,000 gas reverts even with a 1,000,000 gas limit once the maker arms its check. No funds move, but the attacker can repeat this for the cost of one storage write per batch, denying best-effort batching through the `SpectraLimitOrderBatchFiller` contract and the router's `FILL_1INCH_LIMIT_ORDERS` command.

EXPLOIT PATH

1. A contract maker posts an order whose `isValidSignature()` returns valid while a stored flag is unset.
2. The taker simulates and broadcasts a batch via `fillMany()` with `skipReverts` set.
3. The maker sets the flag, so `isValidSignature()` now loops until out of gas.
4. The settlement `staticcall` burns nearly all forwarded gas and reverts; the batch filler lacks the gas to settle remaining fills and refund budgets, so the entire transaction reverts.

RECOMMENDATION

Cap the gas each settlement call can use and do not copy the revert data, so one fill cannot starve the rest of the batch. A per-fill limit chosen by the caller suits fills of different shapes, and the router's copy of the `Fill` struct needs the same field.

Apply the same gas limit and `catch` to the `fillContractOrderArgs()` branch.

RESOLUTION

Spectra

We acknowledge both findings and propose a fix in PR #21:

- **L-01:** skipped fills copy and emit at most the 4-byte error selector, preventing oversized revert data from exhausting the batch's gas.
- **L-02:** each settlement call has a caller-selected gas cap, covering signature checks and hooks.

We forgot to include the corresponding router update in our previous comment: https://github.com/perspectivefi/spectra-router/pull/33.

It adds the trailing `gasLimit` field to the router's `Fill` struct

[L-03] Self-registered fake registry row makes SettlementRouter pay fees from its own balance without collecting them, so anyone can drain tokens held by the router

Low Fixed

SUMMARY

`SettlementRouter._settleCore` pulls the fee through the row's roles modifier and then pays the same amount out from the router's own balance, without checking that the pull actually delivered anything.

`SettlementRouter.sol` L469-476:

```
if (r.totalFee > 0) {
    _execWithRole(
        c.rolesModifier,
        asset_,
        abi.encodeCall(IERC20.transfer, (address(this), r.totalFee))
    );
    _distributeFees(asset_, safe_, recipients, r.mgmtFee, r.perfFee, r.totalFee);
}
```

`_execWithRole` drops the boolean result (L323-332), and `_distributeFees` sends `totalFee` with `SafeERC20.safeTransfer` from the router (L361-364). So the payout happens regardless of whether the pull collected anything.

The settle path never requires the row to come from the factory. The row is resolved by `SafeInfraVaultReader.getInfraVault`, which only checks bindings that the contracts report about themselves:

```
if (IOwnable(vault).owner() != safe) revert UnboundVault(safe, vault);
if (IRole(rolesModifier).avatar() != safe) revert UnboundModifier(safe, rolesModifier);
if (IRole(rolesModifier).target() != safe) revert UnboundModifier(safe, rolesModifier);
```

`MetaVaultsRegistryV2._authorizeWrite` (L187-194) allows a write when `msg.sender == vault` (the row key) and the key is not locked. A fresh row has no locked keys, so any EOA can write a full row under its own address. `docs/SETTLEMENT_ROUTER.md` L61 itself calls the registry "permissionlessly self-writable".

An attacker registers a row keyed to itself, points `core.vault` and `settlement.rolesModifier` at two small contracts that return the attacker from `owner()`, `avatar()` and `target()`, then calls settle. The owner check `msg.sender == c.safe` (L256) passes because the attacker is the row key. The fake modifier returns `true` from `execTransactionWithRole` and executes nothing, so no fee is collected, yet `_distributeFees` still pays `totalFee` from the router balance to a recipient the attacker chose. The PoC moves 300,000e6 of a standard ERC-20 from the router to the attacker.

This differs from the related known issues: M-10 needs a token that returns `false` without reverting on a legitimate vault; here the token is standard and the fake modifier simply moves nothing. L-08 only proposes checking the modifier return value, which does not block a modifier that returns `true`. M-06 assumed no permissionless registry-write path exists, but `_authorizeWrite` is exactly that path.

IMPACT

Any address can take the router's balance of any ERC-20. The attacker names the target token as `asset()` of its fake vault and sets itself as fee recipient. No privileged role, no real Safe or vault, and no non-standard token are needed.

The loss is limited to what the router holds at the time. A legitimate settle of a real vault with a standard asset collects exactly `totalFee` and pays out exactly `totalFee` in one `nonReentrant` call, and the documented invariant `asset.balanceOf(router) == 0 before and after settle` means normal operation leaves nothing behind.

What is exposed is any balance that reaches the router outside this path, for example tokens sent to it by mistake. The router has no sweep, rescue, or admin withdraw function, so such balances sit in the router and anyone can claim them through this path.

EXPLOIT PATH

1. Deploy two small contracts whose `owner()`, `avatar()`, and `target()` all return the attacker address, plus a fake modifier whose `execTransactionWithRole` returns `true` and does nothing.
2. Self-write a registry row keyed to the attacker address, with `core.vault` and `settlement.rolesModifier` pointing at the fake contracts and `asset()` set to the token held by the router.
3. Set the attacker as fee recipient and call settle; the owner check passes because the attacker is the row key.
4. The fee pull executes nothing, but `_distributeFees` transfers `totalFee` of the token out of the router to the attacker.

RECOMMENDATION

Verify the fee actually arrived before paying it out. The modifier return value is not sufficient, since a fake modifier returns `true` while moving nothing. Read the router's balance of `asset_` before and after the fee pull and revert settle if the increase is less than `totalFee`.

Do not settle a self-written row. A lock check does not help, because a self-writer can lock its own keys through the bulk `set` (`MetaVaultsRegistryV2.sol` L110-113). The router should maintain its own list of Safes written by the factory step or governance, and `settle` should revert for any Safe not on that list.

Also add an admin-only sweep so tokens sent to the router by mistake can be recovered.

RESOLUTION

> **Spectra**
>
> The fix in #174 checks the router's asset balance before and after the fee pull, and reverts unless it increased by at least `totalFee`. It also rejects a `false` return from the roles modifier.
>
> This prevents a fake modifier from making the router distribute pre-existing tokens: every payout must be fully funded by the current fee pull. Fee distribution already sends the rounding remainder to the last recipient, so normal settlements leave no fee dust.
>
> We're not adding a Safe whitelist because the balance-delta check directly addresses L-03, including when stray tokens are already present. A whitelist would additionally restrict who can use the router, but it is unnecessary to enforce this invariant and would introduce approval management.

[L-04] SettlementRouter redeposits even a sub-share amount into the IBT and reverts on failure, so a 1-wei deposit request blocks settle, forceSettle and forceSettleWithNav

Low Acknowledged

SUMMARY

Any address can stall every settlement path of a vault with a 1-wei deposit request, provided the IBT rejects deposits that would mint zero shares (such as Aave `ATokenVault` or Yearn V3 `VaultV3`) and the epoch is quiet (no other deposit request and no redeem request).

After settling, the router redeposits any positive asset amount from the AsyncVault preview back into the IBT:

```
(assetsToOwner, , expectedAssetFromOwner, ) = IAmphor(c.vault).previewSettle(newSavedBalance);

if (c.ibt != address(0) && r.assetsToOwner > 0) {
    _execWithRole(
        c.rolesModifier,
        c.ibt,
        abi.encodeCall(IERC4626.deposit, (r.assetsToOwner, c.safe))
    );
}
```

`AsyncVault` sets `assetsToOwner = pendingDeposit - assetsToWithdraw`, where `pendingDeposit` is the asset balance of `pendingSilo`, and `requestDeposit` enforces no minimum. In a quiet epoch, one wei requested by anyone (or sent directly to `pendingSilo`) makes `assetsToOwner = 1`.

Both example IBTs compute shares as `assets * supply / totalAssets` with no virtual offset, so once a share is worth more than one asset unit (the normal state of a yield-bearing IBT) a 1-wei deposit mints zero shares and reverts (`ZERO_SHARES` / "cannot mint zero"). The router sends the deposit with `shouldRevert = true`, so the whole settlement reverts.

This differs from the capacity-based findings: the attacker does not fill a deposit cap with capital, it simply sets the redeposit amount to 1 wei. `previewSettle` still reports success because `_previewIbt` checks only `maxWithdraw`, returning `wouldRevertWith = bytes4(0)` for a settle that actually reverts.

IMPACT

During a quiet epoch, no settlement of any kind executes. The keeper's `settle`, the owners' `forceSettle` and the emergency `forceSettleWithNav` all run `_settleCore` and all revert.

While blocked: no fee is collected, NAV and the high-water mark are not updated, the epoch does not advance, and no oracle round is consumed. The cost to the attacker is 1 wei and one transaction. The wei stays pending until a settle succeeds, so a single request holds the vault until the next honest request arrives, and after each successful settle the attacker needs only one more wei.

Each blocked day adds yield to the next NAV. In the PoC (50 bps cap, NAV starting 30 bps above the 0.3% yield that makes 1 wei buy zero shares, then 6% APR), 30 daily keeper settles fail: 12 on `ZERO_SHARES`, then 18 on `DeviationExceeded`. From there the hot keeper cannot settle even once an honest deposit lifts the veto (NAV +79 bps against the 50 bps cap), and honest deposits and redemptions wait on a cold `forceSettle` from the Safe owners.

Owners can force one settle through with a cold transaction that adds a Safe deposit request worth at least one IBT share, but the attacker re-arms immediately after.

EXPLOIT PATH

1. Wait for (or create) a quiet epoch with no other deposit or redeem request.
2. Call `AsyncVault.requestDeposit` for 1 wei (or transfer 1 wei directly to `pendingSilo`).
3. The next settle previews `assetsToOwner = 1` and attempts `IERC4626.deposit(1, safe)`, which the IBT reverts on zero shares.
4. Every `settle`, `forceSettle` and `forceSettleWithNav` reverts; the vault cannot advance.
5. After any owner-forced settle, repeat with another wei.

RECOMMENDATION

Do not send a redeposit that cannot mint a share. Leave such an amount idle in the Safe, where it still counts toward NAV and covers part of the next fee pull.

```
if (
    c.ibt != address(0) &&
    r.assetsToOwner > 0 &&
    IERC4626(c.ibt).previewDeposit(r.assetsToOwner) > 0
) {
    _execWithRole(
        c.rolesModifier,
        c.ibt,
        abi.encodeCall(IERC4626.deposit, (r.assetsToOwner, c.safe))
    );
}
```

Apply the same skip to the related zero-share deposit case.

RESOLUTION

> Spectra
>
> We acknowledge the edge case, but it requires an unusually small net redeposit amount and is easily recoverable by adding a negligible deposit sufficient to mint an IBT share.
>
> Given the low likelihood and straightforward recovery, we prefer to accept this risk rather than expand the code surface with additional logic.

[L-05] Per-share NAV rescaling truncates aggregate vault value

Low Acknowledged

LOCATION

`SettlementRouter:_rescale and _checkSettle:174-183,293-297`

SUMMARY

When computing aggregate AUM, the router floors the per-share NAV to asset base units in `_rescale` first, then multiplies that truncated value by total supply. Flooring before multiplying discards up to nearly one asset base unit per share, and that loss is never recovered before fees and `newSavedBalance` are computed.

For example, with a 6-decimal asset and an 18-decimal oracle NAV of 1.5 base units per share, `_checkSettle` rescales to 1 base unit per share and drops the 0.5. With one million shares, `_settleCore` then derives AUM from one million units instead of the correct 1.5 million.

If the truncated per-share value rounds to zero, settlement reverts outright even when aggregate vault value is nonzero. This is reachable for any supported asset with fewer than 18 decimals whenever the discarded fraction is nonzero, and it does not require attacker control of the oracle.

IMPACT

Settlement systematically understates AUM, fees, and the vault's saved balance for low-decimal assets, by an amount proportional to share supply. The under-accounting accumulates across settlements.

When the truncated per-share NAV rounds to zero, settlement reverts, blocking the operation despite a nonzero aggregate value.

EXPLOIT PATH

1. The vault asset has fewer than 18 decimals (e.g. 6) and the oracle or explicit NAV carries fractional asset-base-unit precision (e.g. 1.5 units per share).
2. `_checkSettle` rescales the value down to 1 base unit per share, discarding the 0.5 remainder.
3. `_settleCore` computes AUM from the truncated per-share value times supply, understating total value.
4. The understated AUM becomes the settled saved balance and lowers fees; if the truncated per-share NAV is zero, settlement reverts instead.

PREREQUISITES

No privileged role or oracle manipulation is required. The issue triggers whenever the asset has fewer than 18 decimals and the NAV's discarded sub-base-unit remainder is nonzero. It affects both normal oracle settlement and `forceSettleWithNav`.

RECOMMENDATION

Convert precision after the supply multiplication rather than before. Retain the 18-decimal NAV and compute aggregate asset units with a single `mulDiv`:

```
uint8 assetDecimals = IERC20Metadata(asset_).decimals();
uint256 scale = 10 ** (ORACLE_DECIMALS - assetDecimals);
uint256 aum = Math.mulDiv(nav18, supply, scale * c.shareUnit);
```

Pass the untruncated normalized NAV through `_preSettleActions`, `_settleCore`, and the explicit-NAV path, using the same aggregate conversion in each. Validate that aggregate AUM is nonzero rather than rejecting solely because a per-share conversion rounds to zero. Rounding-to-nearest is a reasonable alternative rounding policy.

RESOLUTION

Spectra

For the intended assets and expected NAV range, the potential accounting difference is negligible. For a 6-decimal asset with NAV near 1, truncation introduces an AUM error below 0.0001%, while the underlying assets remain in the Safe.

Preserving 18-decimal precision throughout would require coordinated changes to settlement, fee calculations, previews, and stored NAV benchmarks. We consider that scope disproportionate to the expected impact

[L-06] SettlementRouter stores gross NAV after charging settlement fees

Low Fixed

LOCATION

`SettlementRouter:_settleCore:384-394`

SUMMARY

After a settlement, the router updates its per-vault NAV state (`lastNav` and `highWaterMark`) to the gross oracle value, even though it passes a fee-adjusted balance to `AsyncVault.settle`. The state update never subtracts `totalFee` or derives a post-fee per-share value.

```
st.lastNav = newNav;
if (newNav > st.highWaterMark) {
    st.highWaterMark = newNav;
}
```

This leaves the router's benchmark above the value the vault actually retains after fees. For example, with oracle NAV 100e18 and a fee equal to 1.5% of AUM, the vault settles on a 98.5e18 per-share basis while the router stores `lastNav = 100e18` (and possibly `highWaterMark = 100e18`).

The same gross-versus-net divergence affects `_computeFees`: later fee calculations compare against a value the vault no longer holds.

IMPACT

Routine fee-bearing settlements consume the configured deviation allowance. A later oracle-driven settlement can revert with `DeviationExceeded` even when the real post-fee NAV movement is within the cap, because the decline is measured from the inflated gross baseline.

When high-water-mark mode is disabled, the gross prior NAV becomes the next performance-fee benchmark, overstating it relative to the fee-adjusted vault balance.

When HWM mode is enabled, the gross high-water mark no longer represents the highest post-fee value available to shareholders. This creates a fee-free recovery band below the gross benchmark after every fee-bearing high, and that recovery holiday can recur across multiple fee-bearing settlements. Performance-fee recipients lose fees on gains that merely recover value deducted in a prior fee crystallization, so performance fees are under-collected during recovery.

EXPLOIT PATH

1. A vault settles with oracle NAV 100e18 and `totalFee` equal to 1.5% of AUM; `_settleCore` calls `AsyncVault.settle` on a 98.5e18 per-share basis.
2. The router stores `st.lastNav = 100e18` and may store `highWaterMark = 100e18`.
3. The next oracle reports 97.712e18, a 0.8% decline from the fee-adjusted 98.5e18 value.
4. `_checkSettle` measures a 2.288% decline from `st.lastNav` and reverts with `DeviationExceeded` when the cap is 2%.

PREREQUISITES

A normal settlement must charge a nonzero management or performance fee, and the next oracle-driven settlement must use the interval/deviation path with a deviation cap below the combined fee deduction plus subsequent NAV movement.

RECOMMENDATION

Record the post-fee per-share value in `lastNav` and `highWaterMark` so the stored benchmark matches the basis used by the vault.

```
uint256 postFeeNav = Math.mulDiv(aum - r.totalFee, c.shareUnit, supply);
st.lastNav = postFeeNav;
if (postFeeNav > st.highWaterMark) {
    uint256 oldHwm = st.highWaterMark;
    st.highWaterMark = postFeeNav;
    emit HighWaterMarkUpdated(safe_, oldHwm, postFeeNav);
}
```

Ensure all future fee calculations use this consistently normalized post-fee basis.

RESOLUTION

> Spectra
>
> Acknowledged. The next oracle report reflects assets remaining after the previous settlement's fees, while `lastNav` still stores the pre-fee value. Addressed in PR #176 by storing the post-fee NAV in `lastNav` and updating the HWM on the same basis.

Info Severity

[I-01] The protocol specification rejects the vault share quote kind the contracts accept

Info Fixed

SUMMARY

The normative specification and the contracts disagree on whether vault share quotes are allowed.

`PROTOCOL_SPEC.md` §9.2 defines the rate payload's quote kind as 0 for the underlying and 1 for the IBT, and requires the decoder to reject any other enum value. §8.3 allows exactly four derivative-and-quote combinations, and the §15 validation policy rejects any quote token other than the PT's underlying or IBT.

The contracts go further than the spec. `SpectraPtYtRateAmountGetter._decodeRateData()` accepts quote kind 2 (`_QUOTE_VAULT_SHARE`), `_claimValue()` prices it through the wrapper's `previewUnwrap()` , and `SpectraLimitOrderTokeniser` wraps and unwraps vault shares for it, as the README and the getter's NatSpec describe.

IMPACT

Order book validators and SDKs generated from the normative specification will reject otherwise valid vault share orders, or price them without the unwrap step.

Makers signing such orders have no specification describing how they are priced.

RECOMMENDATION

If vault share quotes are meant to ship, add quote kind 2 to §8.3, §9.2 and §15 of `PROTOCOL_SPEC.md` , including the requirement that the IBT is a wrapper with a nonzero `vaultShare()` and the `previewUnwrap()` claim formula.

If the kind is not meant to ship, reject it in `SpectraPtYtRateAmountGetter._decodeRateData()` instead.

RESOLUTION

> **Spectra**
>
> This docs-only PR adds `quoteKind = 2`, the wrapper requirements and the `previewUnwrap()` pricing formula to the protocol spec: https://github.com/perspectivefi/spectra-lo-otc/pull/22
>
> ---
>
> We just updated again the docs with the missing mentions in the same PR https://github.com/perspectivefi/spectra-lo-otc/pull/22

[I-02] `previewSettle` reports success for settlements that fail the vault's maximum-drawdown check

Info Fixed

SUMMARY

`SettlementRouter.previewSettle()` is meant to tell keepers whether a settlement will revert by returning the expected error selector in `wouldRevertWith`.

The Router only calls `AsyncVault.previewSettle()` inside `_previewIbt()`, which returns early when no IBT is configured:

```
if (c.ibt == address(0)) return r;
```

With no IBT, the vault's maximum-drawdown constraint is never checked, so the preview can return `wouldRevertWith = bytes4(0)` (success) even though the real settlement later reverts with `MaxDrawdownReached()`.

When an IBT is configured, the vault preview runs but its actual error is discarded:

```
try IAmphor(c.vault).previewSettle(newSavedBalance) {
    // ...
} catch {
    r.err = PreviewUnavailable.selector;
    return r;
}
```

`MaxDrawdownReached()` and other vault errors are replaced by the generic `PreviewUnavailable()` selector. Both paths contradict the documented behavior that `previewSettle()` returns the first error the real settlement would encounter.

IMPACT

Keepers may submit settlements that the preview indicated would succeed, only for the real transaction to revert.

This wastes gas, causes repeated failed settlement attempts, delays vault settlement, and complicates operational diagnosis. It does not bypass the drawdown protection or endanger funds, since the real settlement still safely reverts.

RECOMMENDATION

Always call `AsyncVault.previewSettle()` regardless of whether an IBT is configured.

When the call reverts, return the original revert selector in `wouldRevertWith` instead of replacing it with `PreviewUnavailable()`. Use the returned values for IBT-liquidity calculations only when an IBT is present.

RESOLUTION

Spectra

Implemented in PR #178

`previewSettle` now always calls `AsyncVault.previewSettle`, regardless of whether an IBT is configured, and reports the original vault revert selector instead of `PreviewUnavailable`. The returned values are reused for IBT liquidity calculations when set.

[I-03] Fee-charging ERC-4626 IBTs cause Amphor to record more backing than the Safe owns

Info Acknowledged

SUMMARY

`SettlementRouter` assumes that depositing into or withdrawing from the configured ERC-4626 IBT preserves the full economic value of the assets involved. ERC-4626 permits withdrawal and deposit fees, which breaks this assumption.

When the Safe lacks liquid assets, the Router withdraws the exact shortfall:

```
uint256 maxW = IERC4626(c.ibt).maxWithdraw(c.safe);
if (maxW < shortfall) {
    revert InsufficientIbtLiquidity(shortfall, maxW);
}
_execWithRole(
    c.rolesModifier,
    c.ibt,
    abi.encodeCall(IERC4626.withdraw, (shortfall, c.safe, c.safe))
);
```

`withdraw(shortfall, ...)` guarantees the Safe receives `shortfall`, but the IBT may burn shares worth more than `shortfall` to collect the fee. The Router ignores the shares burned and still settles Amphor using the pre-fee value, so Amphor records more remaining backing than the Safe actually owns.

The same problem exists when the Router deposits assets returned by settlement via `IERC4626.deposit(r.assetsToOwner, c.safe)`. A deposit fee can leave the Safe with shares worth less than `assetsToOwner`, while Amphor has already accounted for the complete asset amount.

Neither the deployment pipeline nor the runtime logic restricts the configured IBT to a fee-free implementation.

IMPACT

The discrepancy causes Amphor's `lastSavedBalance` to exceed the Safe's real portfolio value.

For withdrawal fees, redeeming shareholders can exit at the gross reported value while the fee is left for the remaining shareholders to absorb. For deposit fees, new assets are recorded at their full value even though part of that value has already been charged as a fee.

This can:

- Misprice later deposits and redemptions.
- Transfer the IBT fee loss to remaining shareholders.
- Accumulate accounting discrepancies across settlements.
- Cause a later truthful NAV report to fail deviation or maximum-drawdown checks.

RECOMMENDATION

Either restrict settlement IBTs to reviewed, fee-free ERC-4626 implementations or account for their complete deposit and withdrawal costs.

For supported fee-bearing IBTs, use the relevant preview functions and balance/share snapshots, enforce a maximum acceptable fee, and incorporate the actual value lost into Amphor's saved balance. Because deposits occur after Amphor settlement, supporting entry fees may require changing the settlement flow or using a dedicated adapter.

RESOLUTION

> **Spectra**
>
> Acknowledged. Fee-bearing ERC-4626 IBTs are outside the supported configuration for settlement idle liquidity. The configured IBT must use the vault's underlying asset and charge no deposit or withdrawal fees.
>
> We are documenting this requirement in PR #180.

[I-04] Rate getter prices SELL orders with the IBT's full-amount, liquidity-capped `previewRedeem`: a flash-loan taker buys a signed 1,000,000 PT order on an Aave ATokenVault market for under 2 USDC

Info Acknowledged

SUMMARY

`SpectraPtYtRateAmountGetter` prices a SELL order from a single claim read over the whole order size. For an underlying-quoted order that read is `pt.convertToUnderlying(amount)`, which core implements as one `IBT.previewRedeem()` over the full IBT amount (`PrincipalToken.sol` L581-583).

```
fullTaking = SpectraRatePricingLib.sellTaking(_claimValue(pt, quoteKind, amount), d, isYt,
Math.Rounding.Ceil);
if (quoteKind == _QUOTE_UNDERLYING) return pt.convertToUnderlying(amount);
```

Aave's `ATokenVault` caps `previewRedeem` at the cash the Aave reserve holds at that moment (`ATokenVault.sol` L399-402 and L602-615, verified against the wapUSDC implementation `0x907f2aa9a56d7361c49f5d4719bc73bb106bbb73`):

```
uint256 maxWithdrawable = _maxAssetsWithdrawableFromAave(); //
UNDERLYING.balanceOf(address(ATOKEN))
return maxWithdrawable == 0 ? 0 : _convertToAssets(shares,
MathUpgradeable.Rounding.Down).min(maxWithdrawable);
```

Because the getter prices the entire order through one capped preview, the SELL price depends on the reserve's available cash at fill time, and anyone can remove that cash for the length of one transaction. An attacker flash-borrows almost all of it from Aave, fills the maker's order inside the callback, then repays. With the cash gone the getter prices 1,000,000 PT at about 1.96 USDC. Spectra's own `PrincipalToken.flashLoan` opens the same window with no fee, and at high utilisation the cap binds with no flash loan at all.

Core's stored rates are unaffected: `_getCurrentPTandIBTRates` reads the IBT per unit, so as long as the window leaves at least one IBT unit of cash the per-unit rate stays correct (the PoC leaves two units and confirms `updateYield` stores identical rates inside and outside the window). Only the getter uses the full-size preview as a price that someone actually pays. This is also distinct from the known one-unit rounding issue in the same `_claimValue` ; here the price is ~99.9998% too low (1.96 vs 976,339.25).

IMPACT

A maker who signs a public RATE order has no way to protect it. The price is computed by the getter at fill time and the signed `takingAmount` is only a preview, v1 orders cannot carry a predicate for a price floor, and the attack is atomic so no off-chain or risk-disable check can react. The market looks healthy at signing and in any off-chain simulation.

In `test_flashLoan_fillsMillionPtSellOrderForResidualCash` the reserve is healthy with 2.5M USDC of cash. The maker signs SELL 1,000,000 PT for USDC at 6% APY, fair value 976,339.25 USDC. The attacker fills it for 1.96 USDC, pays 1,250 USDC of Aave premium, and walks away with PT worth 1,000,000.97 USDC. Using Spectra's PT flash loan there is no premium (`getPTFlashLoanFee()` returns 0 on both target registries), so the total cost is 1.96 USDC.

This is not limited to whale orders or to the flash-loan path. It applies whenever the PT holds more IBT than the reserve has cash: a 100 PT order against the healthy reserve is exposed, and at 95% utilisation the same 1,000,000 PT order fills at 488,169.15 USDC with no flash loan involved. YT SELL orders flow through the same `_claimValue` call.

Curator Safes are the intended signers of these orders (`docs/SAFE_ORDER_SIGNER.md`), so the loss lands on the metavault's depositors.

This is a formula problem, not a single bad listing. The spec's SELL formula (§8.6) reads one full-size claim, which is only correct if the claim is linear in order size; Spectra's own router already values PTs per unit (`RouterUtil.sol` , `convertToUnderlying(10 ** decimals)`). The spec also requires the getter to work for every PT in the canonical registry (§5.3), and three ATokenVault PTs are registered on Hemi and HyperEVM, both getter targets in `deployments/chains.json` :

chain	PT	IBT	maturity
HyperEVM (999)	0x77a61315aa1ec5fa61fba6226653fe7b09a80f2d	wpUSDT 0xd8d54b633877986f33971540ccd2b385d0e0e93f	2026-11-10
HyperEVM (999)	0x3f6da7ca4bac98dacf00581e860c5eb00e13a7a2	wpUSDC 0x92fd6861ea6f535976be4eed65ca79b2a4e28fa9	2026-11-09
Hemi (43111)	0xe6ba5cf00c46fadbc1ccc42f5167e847acd3ee92	wapUSDC 0x184844cd31cc25f0254b68bdb75288f63b5c5ce4	2026-11-08

The cap has already triggered in the wild with no attacker. Four days after listing, wpUSDC valued the largest holder's 1,041.17 PT at 5.37 USDC while a single PT still read 0.999999:

```
cast call 0x3f6da7ca4bac98dacf00581e860c5eb00e13a7a2 "convertToUnderlying(uint256)(uint256)"
1041167058 --block 19095730  # 5367344
cast call 0x3f6da7ca4bac98dacf00581e860c5eb00e13a7a2 "convertToUnderlying(uint256)(uint256)"
1000000 --block 19095730     # 999999
```

The three registered reserves are almost fully utilised now (cash 0, 0 and 6,733 wei) with maturities of 8-10 Nov 2026.

EXPLOIT PATH

1. A maker (often a Curator Safe) signs a public SELL RATE order for PT on an ATokenVault-backed market that looks healthy.
2. The attacker flash-borrows almost all of the Aave reserve's underlying cash (via Aave, or via Spectra's zero-fee `PrincipalToken.flashLoan`).
3. Inside the callback, with the reserve cash drained, `previewRedeem` is capped near zero, so the getter prices the full order at about 1.96 USDC.
4. The attacker fills the order at that price, receiving PT worth close to its fair value, then repays the flash loan.
5. The maker receives ~1.96 USDC for the entire order; the loss falls on the metavault depositors.

PREREQUISITES

The PT's IBT must be an Aave `ATokenVault` (or any 4626 that caps `previewRedeem` by liquidity), which is the case for three registered PTs on Hemi and HyperEVM. The PT must hold more IBT than the reserve has cash, achieved by a flash loan, or satisfied naturally at high reserve utilisation (where no flash loan is needed).

RECOMMENDATION

Value SELL claims from rates rather than from a full-amount preview that depends on liquidity, and pick whichever value favours the maker:

```
function _claimValue(ISpectraPrincipalToken pt, uint8 quoteKind, uint256 amount, bool isSell)
    private view returns (uint256)
{
    uint256 ibts = pt.previewRedeemForIBT(amount);
```

```
    if (quoteKind == _QUOTE_IBT) return ibts;
    if (quoteKind == _QUOTE_UNDERLYING) {
        uint256 byRate = IERC4626(pt.getIBT()).convertToAssets(ibts); // not capped by
liquidity
        uint256 byPreview = pt.convertToUnderlying(amount);           //
IBT.previewRedeem(ibts)
        return isSell ? Math.max(byRate, byPreview) : Math.min(byRate, byPreview);
    }
    return ISpectra4626Wrapper(pt.getIBT()).previewUnwrap(ibts);
}
```

SELL takes the larger value and BUY the smaller, both in the maker's favour. A simpler alternative is to value per unit like core does (`amount * convertToUnderlying(unit) / unit`), at some precision cost on the BUY probe.

RESOLUTION

> **Spectra**
>
> Our support assumption is ERC-4626 compliance, consistent with Spectra's core protocol. Spectra is permissionless: registration of a PT in the canonical registry does not certify that its IBT complies with ERC-4626.
>
> We agree that these behaviors are problematic for liquidity-capped previews. However, there is no reliable, general on-chain check for ERC-4626 compliance. In particular, rejecting an IBT because `previewRedeem()` returns less than `convertToAssets()` would also reject compliant vaults with withdrawal fees, which we intend to support.
>
> We therefore do not want to introduce heuristic compliance checks or special handling for non-compliant implementations in the getter. The same integration assumption already applies to the core protocol.

[I-05] Rate getter fills IBT-quoted BUY orders at a stale stored rate when the IBT preview is capped, so a free PT flash loan lets the taker deliver less PT than the maker's signed APY

Info Acknowledged

SUMMARY

For a BUY rate order quoted in IBT, the getter values the PT side in IBT with `pt.previewRedeemForIBT(probe)` (`SpectraPtYtRateAmountGetter.sol` L328, L350-351). Core derives the current IBT rate from a one-unit `previewRedeem` and, when that reads below the stored rate, keeps using the stored pair (spectra-core `PrincipalToken.sol` L957-967):

```
_ibtRate = IERC4626(_ibt).previewRedeem(ibtUnit).toRay(underlyingDecimals);
...
_ptRate = _ibtRate < ibtRate ? ptRate.mulDiv(_ibtRate, ibtRate, ...) : ptRate;
```

Aave's `ATokenVault` caps `previewRedeem` at the cash available in the Aave reserve. When that cash falls below one IBT unit, the one-unit preview reads low, core falls back to the stored `ibtRate`, which excludes the yield accrued since the last store, and the getter hands the maker fewer PT than its signed APY.

This fallback path exists in core to handle negative yield. The getter turns it into a fill price, while PROTOCOL_SPEC §8.7 L431 expects the quote to preserve "the signed APY relative to the reduced claim".

Note that patching only the full-amount `convertToUnderlying` read of the underlying branch (L349) does not fix this; the real-core harness applies that patch and the IBT-branch shortfall is unchanged.

IMPACT

The maker of an IBT-quoted BUY order receives less PT than its APY entitles it to, and the attacker pays only gas because the PT flash loan is free (fee 0 on the target registries).

In the transient case the gap equals the IBT yield accrued since the last core rate store. For a BUY of 1M IBT at 6% APY with 5% APR IBT yield, this is 136 ppm one day after the store, 4,092 ppm (4,505 PT) after 30 days, and 18,816 ppm after 140 days.

The gap is far larger once core has already stored a capped rate. Any core call that updates yield (deposit, redeem, claimYield, a YT transfer) while the reserve holds less than one IBT unit of cash writes the capped preview into the stored `ibtRate`. The three ATokenVault PTs registered on Hemi and HyperEVM went through this; the Hemi PT `0xe6ba5cf00c46fadbc1ccc42f5167e847acd3ee92` carries a stored ibtRate of 0.110131 against an IBT rate of 1.558. Once the cash returns the quotes look honest again, yet this same window still falls back to the stale stored rate. In that case the maker pays 1M IBT and receives PT worth 115,534 USDC instead of 1,101,510, an 895,113 ppm shortfall.

EXPLOIT PATH

1. The taker borrows the PT's IBT via Spectra's `PrincipalToken.flashLoan` (zero fee).
2. It withdraws that IBT from the ATokenVault, pushing the Aave reserve cash below one IBT unit for the transaction.
3. The one-unit `previewRedeem` now reads low, so core falls back to the stale stored `ibtRate` when the order is filled on LOP.
4. The taker fills the IBT-quoted BUY order at the understated PT valuation, then deposits back into the vault and repays the flash loan.
5. After the transaction the reserve cash is restored and the quote looks honest again.

PREREQUISITES

Requires an ATokenVault-type IBT whose PT holds enough IBT to drain the reserve cash, plus a public IBT-quoted BUY order to fill. No privileged role is needed.

RECOMMENDATION

Do not accept an IBT quote while the IBT one-unit preview is capped. In the getter, revert with `ZeroRatePrice` when `IBT.previewRedeem(ibtUnit) < IBT.convertToAssets(ibtUnit)`. This prevents core from falling back to the stale stored rate during the fill.

RESOLUTION

> **Spectra**
>
> Our support assumption is ERC-4626 compliance, consistent with Spectra's core protocol. Spectra is permissionless: registration of a PT in the canonical registry does not certify that its IBT complies with ERC-4626.
>
> We agree that these behaviors are problematic for liquidity-capped previews. However, there is no reliable, general on-chain check for ERC-4626 compliance. In particular, rejecting an IBT because `previewRedeem()` returns less than `convertToAssets()` would also reject compliant vaults with withdrawal fees, which we intend to support.
>
> We therefore do not want to introduce heuristic compliance checks or special handling for non-compliant implementations in the getter. The same integration assumption already applies to the core protocol.

[I-06] Conservative ERC-4626 preview can falsely fail tokeniser minOut checks

Info Fixed

LOCATION

`SpectraLimitOrderTokeniser:_redeem:169-174`

SUMMARY

ERC-4626 `previewWithdraw` is an estimate and is permitted to conservatively overstate the shares a withdrawal will burn. The tokeniser uses this estimate to compute the residual IBT left after a fill, then enforces `minOut` against that residual before actually calling `withdraw`.

Because the preview can report more shares than `withdraw` really burns, the computed residual is smaller than the true residual, so a valid fill can fail the minimum check.

```
if (paid == m.underlying) required = IERC4626(m.ibt).previewWithdraw(takingAmount);
if (ibts - required < args.minOut) revert ProceedsBelowMinimum(ibts - required, args.minOut);
```

For example, with 110 IBT shares redeemed, `previewWithdraw(takingAmount)` returning 101 while the actual `withdraw` would burn 100: the code compares `110 - 101 = 9` against `minOut = 10` and reverts, even though execution would leave 10 shares and satisfy the minimum.

IMPACT

Underlying-quoted tokenised fills can revert unnecessarily when a compliant IBT has a conservative preview and the caller sets a tight `minOut`.

This is a liveness and composability issue, not a fund-theft path.

PREREQUISITES

The IBT must be an ERC-4626 implementation whose `previewWithdraw` returns more shares than `withdraw` actually burns, and the caller's `minOut` must fall between the estimated and actual residual amounts. This preview behavior is allowed by the ERC-4626 integration model.

RECOMMENDATION

For the underlying-quoted path, derive the required shares from the actual `withdraw` call rather than from the preview, then apply the `minOut` check against the measured amount:

```
if (paid == m.underlying) {
    uint256 beforeBalance = IERC20(m.ibt).balanceOf(address(this));
    IERC4626(m.ibt).withdraw(takingAmount, taker, address(this));
    uint256 afterBalance = IERC20(m.ibt).balanceOf(address(this));
    uint256 actualRequired = beforeBalance - afterBalance;
    if (ibts < actualRequired) revert InsufficientOutput(ibts, actualRequired);
    if (ibts - actualRequired < args.minOut) revert ProceedsBelowMinimum(ibts
- actualRequired, args.minOut);
} else {
    uint256 required = takingAmount;
    if (paid == m.vaultShare) required = _wrapperSharesFor(m.ibt, takingAmount);
    if (ibts < required) revert InsufficientOutput(ibts, required);
    if (ibts - required < args.minOut) revert ProceedsBelowMinimum(ibts - required,
args.minOut);

    if (paid == m.vaultShare) {
        uint256 vaultShares = ISpectra4626Wrapper(m.ibt).unwrap(required, address(this),
address(this));
        if (vaultShares < takingAmount) revert InsufficientOutput(vaultShares, takingAmount);
        IERC20(m.vaultShare).safeTransfer(taker, takingAmount);
    } else {
        IERC20(m.ibt).safeTransfer(taker, takingAmount);
    }
}
```

RESOLUTION

Spectra

Fixed in https://github.com/perspectivefi/spectra-lo-otc/pull/23

[I-07] Oracle settlement accepts reports older than a manual NAV override

Info Fixed

LOCATION

`SettlementRouter:_checkSettle:255-275`

SUMMARY

Each settlement is meant to advance the vault valuation chronologically. A manual `forceSettleWithNav` operation sets a new NAV and updates `lastSettledAt`, but it does not advance `lastRoundId`. Later

oracle-driven settlement only checks round monotonicity and current staleness, never that the report timestamp is newer than `lastSettledAt`:

```
(roundId, nav18, timestamp) = ISettlementOracle(c.oracle).getLatest();
if (block.timestamp - timestamp > c.maxStaleness)
    revert StaleOracle(timestamp, c.maxStaleness);
if (roundId <= st.lastRoundId)
    revert RoundAlreadyConsumed(roundId, st.lastRoundId);
```

Because the round watermark is left unchanged during the manual override, an oracle report that was published before the override still has a round number above `lastRoundId`. If it remains within the staleness window, it passes both checks and `_settleCore` overwrites the corrected NAV with the obsolete value.

The same gap exists in `SettlementRouter._doSettleWithNav`.

IMPACT

After a manual NAV override, a still-fresh but chronologically older oracle report can roll the vault state backward, replacing a value the Safe deliberately rejected.

The obsolete NAV is then used for AUM, saved-balance, and fee calculations, producing incorrect settlement pricing and fee accounting until a later oracle round replaces it. No privileged role needs to be compromised.

EXPLOIT PATH

1. The oracle publishes round 10 at timestamp 1000 with NAV 1.05e18, while `lastRoundId` is 9.
2. At timestamp 2000, the vault Safe calls `forceSettleWithNav(safe, 0.95e18)`; `lastNav` becomes 0.95e18 and `lastSettledAt` becomes 2000, but `lastRoundId` stays 9.
3. At timestamp 90000, `settle(safe)` reads round 10: round 10 > 9 and its age is below the seven-day `maxStaleness`.
4. Round 10 is accepted even though its timestamp 1000 precedes `lastSettledAt` 2000, and `_settleCore` replaces the corrected 0.95e18 NAV with 1.05e18 and computes fees from the obsolete valuation.

PREREQUISITES

Requires the curator/Safe to have used the manual settlement path, and the older oracle report to still be within the configured staleness window.

RECOMMENDATION

Reject oracle observations that are not newer than the previous settlement timestamp:

```
if (st.lastSettledAt != 0 && timestamp <= st.lastSettledAt) {
    revert StaleOracle(timestamp, c.maxStaleness);
}
```

Also guard against underflow for future-dated oracle timestamps by checking ordering before computing age:

```
if (timestamp > block.timestamp ||
    block.timestamp - timestamp > c.maxStaleness) {
    revert StaleOracle(timestamp, c.maxStaleness);
}
```

RESOLUTION

Spectra

Oracle-driven settlements now require a report timestamp strictly newer than `lastSettledAt`, preventing an older report from being applied after an explicit-NAV settlement. #182